



赛尔瑞 S 系列及 R 系列印卡机 SDK

1.5.3

1. 典型编码和打印步骤	4
1.1 编码磁条卡	4
1.2 编码接触式芯片卡	4
1.3 编码非接智能卡	4
1.4 典型打印	4
1.5 简单打印	4
1.6 双面打印后翻转卡片正面朝上	5
2. 打印机函数	6
2.1 SOY_PR_SdkVersion	6
2.2 SOY_PR_SetLogLevel	6
2.3 SOY_PR_ExecCommand	6
2.4 SOY_PR_GetPrinterInfo	7
2.5 SOY_PR_GetPrinterStatus	9
2.6 SOY_PR_GetPrinterWarning	9
2.7 SOY_PR_GetPrinterConfig	10
2.8 SOY_PR_SetPrinterConfig	10
2.9 SOY_PR_SetStandbyParameters	13
2.10 SOY_PR_WaitPrinterBusy	14
2.11 SOY_PR_ReadTrack	14
2.12 SOY_PR_WriteTrack	15
2.13 SOY_PR_EncodeTrack	15
2.14 SOY_PR_ModifyDocumentProperties	16
2.15 SOY_PR_GetPrinterSetting	17
2.16 SOY_PR_SetPrinterSetting	17
2.17 SOY_PR_GetPrintArea	18
2.18 SOY_PR_SetPrintArea	18
2.19 SOY_PR_DeleteAllJobs	19
2.20 SOY_PR_StartPrinting2	19
2.21 SOY_PR_StartPage2	20
2.22 SOY_PR_EndPage2	20
2.23 SOY_PR_EndPrinting2	20
2.24 SOY_PR_PrintImage2	21
2.25 SOY_PR_PrintText2	21
2.26 SOY_PR_PrintOneCard	22
2.27 SOY_PR_GetBinFirmwareVersion	22
2.28 SOY_PR_UpdateFirmware	23
3. SR30/SR40 智能卡模块函数	24
3.1 SOY_SC_Init	24
3.2 SOY_SC_Exit	24
3.3 SOY_SC_GetVer	24
3.4 SOY_SC_ResetDevice	25
3.5 SOY_SC_Beep	25
3.6 SOY_SC_Read_EEPROM	25
3.7 SOY_SC_Write_EEPROM	26
3.8 SOY_SC_Load_Key	27
3.9 SOY_SC_Hex_A	27
3.10 SOY_SC_A_Hex	27
3.11 SOY_SC_Des	28
3.12 SOY_SC_TripleDes	28
3.13 SOY_SC_Set_CPU	29
3.14 SOY_SC_CpuReset	29
3.15 SOY_SC_Reset	30
3.16 SOY_SC_ConfigCard	30
3.17 SOY_SC_Card_n	31
3.18 SOY_SC_AuthenticationPassAddr	31
3.19 SOY_SC_Write	31
3.20 SOY_SC_Read	32
3.21 SOY_SC_InitVal	32
3.22 SOY_SC_ReadVal	33
3.23 SOY_SC_Increment	33
3.24 SOY_SC_Decrement	34

3.25	<i>SOY_SC_Restore</i>	34
3.26	<i>SOY_SC_Transfer</i>	34
3.27	<i>SOY_SC_Pro_ResetInt</i>	35
3.28	<i>SOY_SC_Pro_CommandLinkInt</i>	35
3.29	<i>SOY_SC_Card_B</i>	36
3.30	<i>SOY_SC_Pro_Reset</i>	36
3.31	<i>SOY_SC_MF_PLUS_WritePerson</i>	36
3.32	<i>SOY_SC_Auth_Ulc</i>	37
3.33	<i>SOY_SC_VerifyPin_4442</i>	37
3.34	<i>SOY_SC_Write_4442</i>	38
3.35	<i>SOY_SC_Read_4442</i>	38
3.36	<i>SOY_SC_VerifyPin_4428</i>	38
3.37	<i>SOY_SC_Write_4428</i>	39
3.38	<i>SOY_SC_Read_4428</i>	39
3.39	<i>SOY_SC_Write_24c</i>	40
3.40	<i>SOY_SC_Read_24c</i>	40
3.41	<i>SOY_SC_CpuApuInt</i>	41
3.42	<i>SOY_SC_Card</i>	41
3.43	<i>SOY_SC_CpuApu</i>	41
3.44	<i>SOY_SC_Pro_CommandLink</i>	42
4.	UHF 模块函数	43
4.1	<i>UhfReaderConnect</i>	43
4.2	<i>UhfReaderDisconnect</i>	43
4.3	<i>UhfGetPower</i>	43
4.4	<i>UhfSetPower</i>	44
4.5	<i>UhfSetFrequency</i>	44
4.6	<i>UhfReadDataByEPC</i>	45
4.7	<i>UhfReadDataByTID</i>	45
4.8	<i>UhfInventorySingleTag</i>	46
4.9	<i>UhfAddFilter</i>	46
4.10	<i>UhfWriteDataByEPC</i>	47
5.	结构.....	48
5.1	<i>SEAORY_DOC_PROP</i>	48
5.2	<i>PRINT_CARD_PARAM</i>	50
6.	常量.....	53
6.1	打印机错误代码	53
6.2	打印机警告代码	55
6.3	SR30/SR40 智能卡读卡器错误代码.....	55
6.4	打印机函数可能返回的 Win32 错误代码.....	56

1. 典型编码和打印步骤

1.1 编码磁条卡

要对磁条卡进行编码, 请按以下步骤调用 SDK 功能:

1. 调用 [SOY_PR_WriteTrack\(\)](#) 将数据写入磁性轨道。打印机将自动进卡。
2. 调用 [SOY_PR_ReadTrack\(\)](#) 读取磁性轨道上的数据。通过代码检查编码结果。
3. 编码后, 您可以通过调用 [SOY_PR_ExecCommand\(\)](#) 带参数 `CMD_MOVE_CARD_TO_HOPPER`, 或移动卡到其他编码站。或者直接开始打印程序。

1.2 编码接触式芯片卡

要对接触式芯片卡进行编码, 请按以下步骤调用 SDK 功能:

1. 调用 [SOY_PR_ExecCommand\(\)](#) 带参数 `CMD_MOVE_CARD_TO_CONTACT` 移动卡到接触式编码站。
2. 调用 [SOY_SC_Xxx\(\)](#) 函数执行编码。
3. 编码后, 您可以通过调用 [SOY_PR_ExecCommand\(\)](#) 带参数 `CMD_MOVE_CARD_TO_HOPPER`, 或移动卡到其他编码站。或者直接开始打印程序。

1.3 编码非接智能卡

要对非接触式智能卡进行编码, 请按以下步骤调用 SDK 功能:

1. 调用 [SOY_PR_ExecCommand\(\)](#) 带参数 `CMD_MOVE_CARD_TO_CONTACTLESS` 移动卡到非接编码站。
2. 调用 [SOY_SC_Xxx\(\)](#) 函数执行编码。
3. 编码后, 您可以通过调用 [SOY_PR_ExecCommand\(\)](#) 带参数 `CMD_MOVE_CARD_TO_HOPPER`, 或移动卡到其他编码站。或者直接开始打印程序。

1.4 典型打印

使用 Windows GDI 函数的典型打印过程如下所示:

1. 调用 `CreateDC()` 创建目标打印机的打印机 DC。
2. 可选, 调用 `DocumentProperties()`或调用 [SOY_PR_ModifyDocumentProperties\(\)](#) 以修改打印作业设置。如果您不调用这二个函数, 该作业将使用默认打印设置。
3. 调用 `StartDoc()` 开始打印作业。
4. 调用 `StartPage()` 开始新页面。
5. 调用 `StretchDIBits/StretchBlt/TextOut` 和其他函数等, 在打印机 DC 上绘制。
6. 调用 `EndPage()` 结束页面。
7. 通过执行步骤 4 ~ 6 打印其他页面。
8. 调用 `EndDoc()` 结束此打印作业。
9. 调用 `DeleteDC()` 释放打印机 DC。

1.5 简单打印

使用 Seaory SDK 简单打印函数的打印过程如下所示:

1. 调用 `SOY_PR_StartPrinting2()` 初始化打印工作, 可选择带入

SEAORY_DOC_PROP 以修改打印作业设置, 或不带入 SEAORY_DOC_PROP 则会使用默认打印首选项。

2. 调用 SOY_PR_StartPage2() 开始新页面。
3. 可选, 调用 SOY_PR_PrintText2() 打印文字。
4. 可选, 调用 SOY_PR_PrintImage2() 打印图档。
5. 调用 SOY_PR_EndPage2() 结束页面。
6. 通过执行步骤 2 ~ 5 打印其他页面。
7. 调用 SOY_PR_EndPrinting2() 结束此打印工作。

1.6 双面打印后翻转卡片正面朝上

完成双面打印后, 如果需要将卡的正面翻转朝上, 请执行以下步骤:

1. 初始化打印工作时, 请带入 SEAORY_DOC_PROP, 修改 byOutputBin=4 以修改打印作业设置。或不带入 SEAORY_DOC_PROP, 直接修改打印首选项->打印设置->出卡->选"不要排出卡片"。
2. 按照 1.4 或 1.5 过程打印。
3. 持续调用 SOY_PR_GetPrinterStatus() 直到打印完成。
4. 调用 SOY_PR_ExecCommand(), 带入命令 CMD_FLIP_CARD, 让卡片翻转。
5. 调用 SOY_PR_ExecCommand(), 带入命令 CMD_MOVE_CARD_TO_HOPPER 或其他动卡命令, 将卡片排出。

2. 打印机函数

2.1 SOY_PR_SdkVersion

读取 SDK 版本号。

语法

```
uint32_t __stdcall SOY_PR_SdkVersionW(WCHAR* szVersionOutW);  
uint32_t __stdcall SOY_PR_SdkVersionA(char* szVersionOutA);
```

参数

szVersionOut [out] 缓冲区的地址, 以接收 SDK 版本号。

返回值

如果函数执行成功, 返回值为 0. 否则返回值为 WIN32 错误代码.

2.2 SOY_PR_SetLogLevel

设置是否启用调用函数的日志。

语法

```
void __stdcall SOY_PR_SetLogLevel(uint32_t dwLogLevel);
```

参数

dwLogLevel [in] 0 表示关闭日志, 1 表示要启用日志。

返回值

没有返回值。

注:

Windows 系统, 日志路径为 C:\Seaory\yyyymmdd.log

Linux 系统, 日志路径为 /tmp/Seaory/yyyymmdd.log

2.3 SOY_PR_ExecCommand

执行打印机命令。

语法

```
uint32_t __stdcall SOY_PR_ExecCommandW(const WCHAR* szPrinterNameW, uint32_t  
dwCommand);
```

uint32_t __stdcall SOY_PR_ExecCommandA(const char* szPrinterNameA, uint32_t dwCommand);

参数

szPrinterName [in] 打印机队列名称。

dwCommand [in] 指定要执行的打印机命令。

dwCommand 的名称	数值	描述
CMD_MOVE_CARD_TO_HOPPER	1	移动卡片到出卡盒(后端)
CMD_MOVE_CARD_TO_CONTACT	2	移动卡片到接触式芯片编码站
CMD_MOVE_CARD_TO_CONTACTLESS	3	移动卡片到非接编码站
CMD_MOVE_CARD_TO_STANDBY_BACK	4	移动卡片到后端待取位置
CMD_MOVE_CARD_TO_FLIPPER	5	移动卡片到翻转器
CMD_MOVE_CARD_TO_REJECT_BOX_FRONT	6	移动卡片到前侧废卡槽
CMD_MOVE_CARD_TO_REJECT_BOX_DOWN	7	移动卡片到下方废卡槽
CMD_MOVE_CARD_TO_STANDBY_FRONT	8	移动卡片到前端待取位置
CMD_MOVE_CARD_TO_FRONT	9	移动卡片到前端位置
CMD_MOVE_CARD_TO_PREPARE	10	移动卡片到准备位置
CMD_RESET_PRINTER_HARD	21	打印机复位
CMD_RESET_PRINTER_JAM	22	打印机有卡复位
CMD_CLEAN_CARD_PATH	23	清洁卡片路径
CMD_FLIP_CARD	24	翻转卡片
CMD_CLEAN_CARD_PATH_NO_EJECT	25	清洁卡片路径，清洁完成后不会排出卡片
CMD_INITIALIZE_FLIPPER	27	初始化翻转器
CMD_ERASE_FULL_CARD	31	抹除整张卡片(可复写卡)

返回值

如果函数执行成功,返回值为 0. 否则返回值为 WIN32 错误代码或打印机错误代码.

2.4 SOY_PR_GetPrinterInfo

获取打印机信息。例如打印机型号名称、固件版本和序列号。

语法

uint32_t __stdcall SOY_PR_GetPrinterInfoW(const WCHAR* szPrinterNameW, int32_t nInfoType, WCHAR* szInfoOutW);

uint32_t __stdcall SOY_PR_GetPrinterInfoA(const char* szPrinterNameA, int32_t nInfoType, WCHAR* szInfoOutW);

参数

szPrinterName [in] 打印机队列名称。

nInfoType [in] 打印机信息类型.

nInfoType 的名称	数值	szInfoOut 的描述	缓冲区返回的值
INFO_MODEL_NAME	1	型号名称	
INFO_FW_VERSION	2	固件版本	
INFO_SERIAL_NUMBER	3	打印机序列号	
INFO_CARD_POSITION	4	卡片位置	"0":Out of printer 无卡 "3":Front standby 前端待取卡位置 "4":Flipper 翻转器位置 "5":Mag in 读磁开始位置 "6":Mag out 读磁结束位置 "7":Start printing 开始打印位置 "8":Print end 结束打印位置 "9":Contact 接触智能卡位置 "10":Contactless 非接智能卡位置 "11":Back standby 后端待取卡位置 "12":Card jam pos 夹卡位置 "13":Prepare pos 准备位置 "15":Unknow pos 未知位置
INFO_RIBBON_TYPE	5	色带类型	
INFO_RIBBON_COUNT	6	色带余量	彩色带返回实际可打印的面数。 单色带是按长度计算, 如规格为100米长度。新色带的初始值为105000, 单位为毫米, 相当于初始值为105米 (为了让色带能使用完毕, 实际写入的长度值要大一些)。当获取到黑色余量为104522, 表示余量为104522毫米, 约等于104米。打印首选项和证卡机显示屏里显示的容量值为 H (表示余量高, 表示长度大于75%), M (表示余量中等, 小于75%, 大于25%), L(表示余量低, 长度小于25%)
INFO_PRINT_COUNT	7	打印计数	
INFO_REGION_CODE	8	区码	
INFO_UNCLEAN_COUNT	9	未清洁计数	
INFO_INSTALLED_MODULE	10	已安装模块·请见下表	
INFO_REJECT_BOX_STATUS	11	废卡盒状态	请参考示例 位与(&)0x00000001表示废卡盒盖是否打开 位与(&)0x00000002表示废卡盒是否已满
INFO_CARD_OUT_STATUS	12	无卡状态	"0" 表示有卡 "1" 表示无卡
INFO_FILM_COUNT	13	转印膜余量	
INFO_MAC_ADDRESS	14	以太网 MAC 地址	

可安装的模块的旗标	数值	模块的描述
MODULE_FLIPPER	0x00000001	翻转器模块
MODULE_MAG_STRIPE	0x00000002	磁卡模块
MODULE_CONTACT_IC	0x00000004	接触式芯片模块
MODULE_RFID	0x00000008	非接触式芯片模块
MODULE_ETHERNET	0x00000010	网络模块
MODULE_600DPI	0x00010000	600 DPI 打印功能

返回值

如果函数执行成功,返回值为 0。否则返回值为 WIN32 错误代码。

2.5 SOY_PR_GetPrinterStatus

获取打印机状态。

语法

```
uint32_t __stdcall SOY_PR_GetPrinterStatusW(const WCHAR* szPrinterNameW,
uint32_t *lpdwStatus);
uint32_t __stdcall SOY_PR_GetPrinterStatusA(const char* szPrinterNameA, uint32_t
*lpdwStatus);
```

参数

szPrinterName [in] 打印机队列名称。

lpdwStatus [out] 接收打印机状态的缓冲区的地址。

零表示打印机已就绪, 没有错误。

其他值表示打印机处于错误状态。请参阅打印机错误代码。

返回值

如果函数执行成功,返回值为0。否则返回值为 WIN32 错误代码。

2.6 SOY_PR_GetPrinterWarning

获取打印机警告状态。

语法

```
uint32_t __stdcall SOY_PR_GetPrinterWarningW(const WCHAR* szPrinterNameW,
uint32_t *lpdwWarning);
uint32_t __stdcall SOY_PR_GetPrinterWarningA(const char* szPrinterNameA, uint32_t
```

* lpdwWarning);

参数

szPrinterName [in] 打印机队列名称。

lpdwWarning [out] 缓冲区的地址, 以接收打印机警告状态。
零表示打印机已准备就绪, 无需警告。
其他值表示打印机有警告. 请参阅打印机警告代码。

返回值

如果函数执行成功, 返回值为 0。 否则返回值为 WIN32 错误代码。

2.7 SOY_PR_GetPrinterConfig

取得当前打印机配置的值。

语法

```
uint32_t __stdcall SOY_PR_GetPrinterConfigW(const WCHAR* szPrinterNameW,  
uint32_t dwConfigType, int32_t* lpnConfigValue);  
uint32_t __stdcall SOY_PR_GetPrinterConfigA(const char* szPrinterNameA, uint32_t  
dwConfigType, int32_t* lpnConfigValue);
```

参数

szPrinterName [in] 打印机队列名称。

dwConfigType [in] 要取得的打印机配置。

lpnConfigValue [in,out] 接收打印机配置的缓冲区的地址。
对于 CONFIG_INPUT_OUTPUT_BINDING, 输入值应为 0 或 1 或 2。
对于 CONFIG_OUTPUT_BIN_MODE, 输入值应为 1 或 2。

返回值

如果函数执行成功, 返回值为 0。 否则返回值为 WIN32 错误代码。

2.8 SOY_PR_SetPrinterConfig

将打印机配置设置为指定的新值。

语法

```
uint32_t __stdcall SOY_PR_SetPrinterConfigW(const WCHAR* szPrinterNameW,  
uint32_t dwConfigType, int32_t nConfigValue);  
uint32_t __stdcall SOY_PR_SetPrinterConfigA(const char* szPrinterNameA, uint32_t  
dwConfigType, int32_t nConfigValue);
```

参数

szPrinterName [in] 打印机队列名称。

dwConfigType [in] 要改变的打印机配置。

nConfigValue [in] 打印机配置的新值。

dwConfigType 的名称	数值	描述	nConfigValue 的值
CONFIG_MANUAL_FEED_FRONT	1	前端手动进卡口	0=禁用 1=启用检测到卡片时立即自动进卡
CONFIG_MANUAL_FEED_BACK	2	后端手动进卡口	0=禁用 1=启用检测到卡片时立即自动进卡
CONFIG_INPUT_BIN	3	设置进卡来源	0=自动进卡器 1=前端手动进卡口 2=后端手动进卡口
CONFIG_ADF_HOOK_MODE	4	设置是否使用勾卡模式	0=禁用勾卡模式 1=勾卡模式 2=以勾卡模式重试进卡 3=勾卡模式+以勾卡模式重试进卡
CONFIG_OUTPUT_BIN_MODE	5	出卡模式	格式为 0xYYYYZZZZ YYYY=出卡位置: 0000=出卡盒 0001=前端手动进卡口 0002=后端手动进卡口 ZZZZ=出卡模式: 0000=直接出卡 0001=走到待取卡位置
CONFIG_INPUT_OUTPUT_BINDING	6	进卡出卡绑定	格式为 0xSSSSTTTT SSSS=进卡来源: 0000=自动进卡器 0001=前端手动进卡口 0002=后端手动进卡口 TTTT=出卡位置: 0000=出卡盒 0001=前端手动进卡口 0002=后端手动进卡口 0003=废卡盒 0004=不要排出卡片
CONFIG_CARD_IN_SIGNAL	7	设置插卡电平	0=正常 (插卡时为低电平) 1=反向 (插卡时为高电平)
CONFIG_REJECT_BOX_SENSOR	8	设置废卡盒传感器检测模式	位0x00000001用于废卡盒盖传感器，位0x00000002用于废卡盒满传感器。 当该位设置为1时，表示打印机固件在尝试将卡移至废卡盒时才会检测传感器。否则，当打印机状态准备就绪时，打印机固件就会检测传感器。
CONFIG_CARD_OUT_SENSOR	9	设置无卡侦测传感器	0=禁用。 1=启用无卡侦测传感器
CONFIG_REJECT_BIN	10	设置排废卡位置	0=出卡盒, 1=前端手动进卡口, 2=后端手动进卡口, 3=废卡盒。

dwConfigType 的名称	数值	描述	nConfigValue 的值
CONFIG_INITIAL_RIBBON_MODE	11	设置初始色带模式	0=于初始化动作时初始色带 1=于第一份工作时初始色带
CONFIG_IC_CARD_POS	12	设置移动到接触编码器的卡位置	默认值为 0, 设置范围为 -100~+100。
CONFIG_RF_CARD_POS	13	设置移动到非接触式编码器的卡位置	默认值为 0, 设置范围为 -20~+20。
CONFIG_PRINT_POS_L	14	设置打印卡片起印端的偏移量	默认值为 0。范围为 -15 ~ +15。
CONFIG_PRINT_POS_T	15	设置打印卡片尾端的偏移量	默认值为 0。范围为 -15 ~ +15。
CONFIG_PRINT_POS_S	16	设置打印卡片侧边的偏移量	默认值为 0。范围为 -15 ~ +15。
CONFIG_STANDBY_FRONT_END	17	设置前端待取卡参数	0xPPPPTTTT PPPP 为位置，单位是毫米。范围为 0 ~ +80。 TTTT 为时间，单位是 0.5秒。范围为 0 ~ +120000。
CONFIG_STANDBY_BACK_END	18	设置后端待取卡参数	0xPPPPTTTT PPPP 为位置，单位是毫米。范围为 0 ~ +80。 TTTT 为时间，单位是 0.5秒。范围为 0 ~ +120000。
CONFIG_AUTO_EJECT_AT_INIT	19	开机或硬重置打印机时自动弹出卡	0=禁用, 1=启用
CONFIG_AUTO_EJECT_OUTPUT_BIN	20	设置自动弹出卡位置	0=出卡盒, 1=前端手动进卡口, 2=后端手动进卡口, 3=废卡盒。

dwConfigType 的名称	数值	描述	nConfigValue 的值
CONFIG_RTR_CARD_TYPE	101	转印要使用的参数组	0=标准 PVC 卡 1=用户定义卡 1 2=用户定义卡 2 3=用户定义卡 3
CONFIG_RTR_USER1_SPEED	102	用户定义卡 1 的转印速度	默认为 0。范围为 -10~+10。
CONFIG_RTR_USER1_TEMPERATURE	103	用户定义卡 1 的转印温度	默认为 0。范围为 -11~+7。
CONFIG_RTR_USER1_OFFSET	104	用户定义卡 1 的转印偏移量	默认为 0。范围为 0~10。
CONFIG_RTR_USER2_SPEED	105	用户定义卡 2 的转印速度	默认为 0。范围为 -10~+10。
CONFIG_RTR_USER2_TEMPERATURE	106	用户定义卡 2 的转印温度	默认为 0。范围为 -11~+7。
CONFIG_RTR_USER2_OFFSET	107	用户定义卡 2 的转印偏移量	默认为 0。范围为 0~10。
CONFIG_RTR_USER3_SPEED	108	用户定义卡 3 的转印速度	默认为 0。范围为 -10~+10。
CONFIG_RTR_USER3_TEMPERATURE	109	用户定义卡 3 的转印温度	默认为 0。范围为 -11~+7。
CONFIG_RTR_USER3_OFFSET	110	用户定义卡 3 的转印偏移量	默认为 0。范围为 0~10。
CONFIG_RTR_FLAT_SINGLE_SIDE	111	单面打印时的整平卡选项	0=默认模式 1=不整平 2=整平一次 3=整平二次 4=整平三次
CONFIG_RTR_FLAT_FRONT_SIDE	112	双面打印时正面的整平卡选项	
CONFIG_RTR_FLAT_BACK_SIDE	113	双面打印时背面的整平卡选项	
CONFIG_RTR_FLAT_SPEED	114	整平卡速度	默认为 0。范围为 0~200。
CONFIG_RTR_EJECT_CARD_SIDE	115	出卡卡面选项	0=默认 1=和进卡卡面相同 2=和进卡卡面相反

返回值

如果函数执行成功,返回值为 0。否则返回值为 WIN32 错误代码。

2.9 SOY_PR_SetStandbyParameters

设置打印机待取卡参数。

语法

```
uint32_t __stdcall SOY_PR_SetStandbyParametersW(const WCHAR* szPrinterNameW,
uint8_t byOutputBin, uint8_t byStandbyPos, uint32_t dwStandbyTime);
uint32_t __stdcall SOY_PR_SetStandbyParametersA(const char* szPrinterNameA,
uint8_t byOutputBin, uint8_t byStandbyPos, uint32_t dwStandbyTime);
```

参数

<i>szPrinterName</i>	[in] 打印机队列名称。
<i>byOutputBin</i>	[in] 出卡方向。1=前端, 2=后端。
<i>byStandbyPos</i>	[in] 卡片伸出的长度。单位是毫米。
<i>dwStandbyTime</i>	[in] 卡片停留时间。单位是 0.5 秒。

返回值

如果函数执行成功,返回值为 0。否则返回值为 WIN32 错误代码。

2.10 SOY_PR_WaitPrinterBusy

等到打印机不处于忙碌状态，然后获取打印机状态。

语法

```
uint32_t __stdcall SOY_PR_WaitPrinterBusyW(const WCHAR* szPrinterNameW,
uint32_t *lpdwStatus);
uint32_t __stdcall SOY_PR_WaitPrinterBusyA(const char* szPrinterNameA, uint32_t
*lpdwStatus);
```

参数

<i>szPrinterName</i>	[in] 打印机队列名称。
<i>lpdwStatus</i>	[out] 接收打印机状态的缓冲区的地址。 零表示打印机已就绪, 没有错误。 其他值表示打印机处于错误状态。请参阅打印机错误代码。

返回值

如果函数执行成功,返回值为 0。否则返回值为 WIN32 错误代码。

2.11 SOY_PR_ReadTrack

读取磁条轨道数据。

语法

```
uint32_t __stdcall SOY_PR_ReadTrackW(const WCHAR* szPrinterNameW, uint32_t
dwMode, WCHAR* szTrack1W, WCHAR* szTrack2W, WCHAR* szTrack3W);
uint32_t __stdcall SOY_PR_ReadTrackA(const char* szPrinterNameA, uint32_t
dwMode, char * szTrack1A, char * szTrack2A, char * szTrack3A);
```

参数

<i>szPrinterName</i>	[in]打印机队列名称。
----------------------	--------------

dwMode [in] 不使用。为将来预留。

szTrack1 [out] 缓冲区的地址, 以接收磁道1数据.可以为 NULL。

szTrack2 [out] 缓冲区的地址, 以接收磁道2数据.可以为 NULL。

szTrack3 [out] 缓冲区的地址, 以接收磁道3数据.可以为 NULL。

返回值

如果函数执行成功,返回值为 0。否则返回值为 WIN32 错误代码或打印机错误代码。

2.12 SOY_PR_WriteTrack

写入数据到磁条轨道。

语法

```
uint32_t __stdcall SOY_PR_WriteTrackW(const WCHAR* szPrinterNameW, uint32_t dwMode, const WCHAR* szTrack1W, const WCHAR* szTrack2W, const WCHAR* szTrack3W);  
uint32_t __stdcall SOY_PR_WriteTrackA(const char* szPrinterNameA, uint32_t dwMode, const char * szTrack1A, const char * szTrack2A, const char * szTrack3A);
```

参数

szPrinterName [in]打印机队列名称。

dwMode [in] 值为0表示以低抗写磁。值为1表示以高抗写磁。

szTrack1 [in] 用于设置要写入的轨道1数据的缓冲区的地址。如果轨道没有要写入的数据，则可以为 NULL。

szTrack2 [in] 用于设置要写入的轨道2数据的缓冲区的地址。如果轨道没有要写入的数据，则可以为 NULL。

szTrack3 [in] 用于设置要写入的轨道3数据的缓冲区的地址。如果轨道没有要写入的数据，则可以为 NULL。

返回值

如果函数执行成功,返回值为 0。否则返回值为 WIN32 错误代码或打印机错误代码。

2.13 SOY_PR_EncodeTrack

写入数据到磁条轨道，然后读回轨道数据以检查写磁是否成功。

语法

```
uint32_t __stdcall SOY_PR_EncodeTrackW(const WCHAR* szPrinterNameW, uint32_t dwMode, const WCHAR* szTrack1W, const WCHAR* szTrack2W, const WCHAR*
```

```
szTrack3W , uint32_t dwRetry);
uint32_t __stdcall SOY_PR_EncodeTrackA(const char* szPrinterNameA, uint32_t
dwMode, const char * szTrack1A, const char * szTrack2A, const char * szTrack3A ,
uint32_t dwRetry);
```

参数

szPrinterName [in] 打印机队列名称。

dwMode [in] 值为0表示以低抗写磁。值为1表示以高抗写磁。

szTrack1 [in] 用于设置要写入的轨道1数据的缓冲区的地址。如果轨道没有要写入的数据，则可以为 NULL。

szTrack2 [in] 用于设置要写入的轨道2数据的缓冲区的地址。如果轨道没有要写入的数据，则可以为 NULL。

szTrack3 [in] 用于设置要写入的轨道3数据的缓冲区的地址。如果轨道没有要写入的数据，则可以为 NULL。

dwRetry [in] 设置编码失败时重试的次数。 0表示写入轨道一次。

返回值

如果函数执行成功,返回值为 0. 否则返回值为 WIN32 错误代码。

2.14 SOY_PR_ModifyDocumentProperties

修改打印作业设置而不打开驱动程序介面。(只限 Windows 系统)

语法

```
uint32_t __stdcall SOY_PR_ModifyDocumentPropertiesW(const WCHAR*
szPrinterNameW, HDC hDC, uint8_t* lpInDocProp, uint8_t* lpInDevMode);
uint32_t __stdcall SOY_PR_ModifyDocumentPropertiesA(const char* szPrinterNameA,
HDC hDC, uint8_t* lpInDocProp, uint8_t* lpInDevMode);
```

参数

szPrinterName [in]打印机队列名称。

hDC [in] 返回的打印机 DC 句柄 CreateDC().

lpInDocProp [in][SEAORY_DOC_PROP](#) 结构的指针, 指定工作要更改的设置.

lpInDevMode [in] DEVMODE 结构的指针, 您的软件在打印作业之前得到的打印机设置。可以为 NULL, 则此函数将使用当前的默认打印机设置。

返回值

如果函数执行成功,返回值为 0. 否则返回值为 WIN32 错误代码。

言论

此函数将打印机设置与 [SEAORY_DOC_PROP](#) 结构作合并。然后它会调用 [ResetDC\(\)](#) 将更改应用于打印机。

如果要使用 [SEAORY_DOC_PROP](#) 中未定义的某些设置, 请通过打印机 UI 进行更改设

置, 然后传给参数 `lpInDevMode`.

2.15 SOY_PR_GetPrinterSetting

读取当前用户的打印首选项, 然后对应到结构 `SEAORY_DOC_PROP`。(只限 Windows 系统)

语法

```
uint32_t __stdcall SOY_PR_GetPrinterSettingW(const WCHAR* szPrinterNameW,  
uint8_t* lpDocPropOut);  
uint32_t __stdcall SOY_PR_GetPrinterSettingA(const char* szPrinterNameA, uint8_t*  
lpDocPropOut);
```

参数

szPrinterName [in] 打印机队列名称。

lpDocPropOut [out] 指定接收打印首选项的结构 `SEAORY_DOC_PROP` 的指针。

返回值

如果函数执行成功, 返回值为 0. 否则返回值为 WIN32 错误代码.

2.16 SOY_PR_SetPrinterSetting

将结构 `SEAORY_DOC_PROP` 设置到当前用户的打印首选项。(只限 Windows 系统)

语法

```
uint32_t __stdcall SOY_PR_SetPrinterSettingW(const WCHAR* szPrinterNameW,  
uint8_t* lpDocPropIn);  
uint32_t __stdcall SOY_PR_SetPrinterSettingA(const char* szPrinterNameA, uint8_t*  
lpDocPropIn);
```

参数

szPrinterName [in] 打印机队列名称。

lpDocPropOut [in] 指定要设置到打印首选项的结构 `SEAORY_DOC_PROP` 的指针。

返回值

如果函数执行成功, 返回值为 0. 否则返回值为 WIN32 错误代码.

2.17 SOY_PR_GetPrintArea

读取当前用户的打印首选项其中的"打印区域"。(只限 Windows 系统)

语法

```
uint32_t __stdcall SOY_PR_GetPrintAreaW(const WCHAR* szPrinterNameW, int32_t nOrientation, int32_t nSide, int32_t nPanel, int32_t* lpnAreaNum, int32_t* lpcbBytes, WCHAR* lpAreaNameAll, WCHAR* lpAreaNameDefault);  
uint32_t __stdcall SOY_PR_GetPrintAreaA(const char* szPrinterNameA, int32_t nOrientation, int32_t nSide, int32_t nPanel, int32_t* lpnAreaNum, int32_t* lpcbBytes, char* lpAreaNameAll, char* lpAreaNameDefault);
```

参数

<i>szPrinterName</i>	[in] 打印机队列名称。
<i>nOrientation</i>	[in] 1 = 纵向 · 2 = 横向。
<i>nSide</i>	[in] 0 = 正面 · 1 = 背面。
<i>nPanel</i>	[in] 0 = YMCO · 1 = K · 2 = Rewrite。
<i>lpnAreaNum</i>	[out] 返回打印区域个数。
<i>lpcbBytes</i>	[out] 返回所有打印区域名称所需的内存大小。
<i>lpAreaNameAll</i>	[out] 缓冲区的地址, 以接收所有打印区域名称。这是一系列以空字符结尾的字符串, 以空字符串 (\0) 终止。以下是一个示例: Area1\0Area2\0Area3\0LastArea\0\0
<i>lpAreaNameDefault</i>	[out] 缓冲区的地址, 以接收默认的打印区域名称。

返回值

如果函数执行成功,返回值为 0. 否则返回值为 WIN32 错误代码。

2.18 SOY_PR_SetPrintArea

将保存的"打印区域"之一设置为当前用户的打印首选项的默认打印区域。(只限 Windows 系统)

语法

```
uint32_t __stdcall SOY_PR_SetPrinterSettingW(const WCHAR* szPrinterNameW, int32_t nOrientation, int32_t nSide, int32_t nPanel, WCHAR* szAreaNameDefault);  
uint32_t __stdcall SOY_PR_SetPrinterSettingA(const char* szPrinterNameA, int32_t nOrientation, int32_t nSide, int32_t nPanel, char* szAreaNameDefault);
```

参数

<i>szPrinterName</i>	[in] 打印机队列名称。
<i>nOrientation</i>	[in] 1 = 纵向 · 2 = 横向。
<i>nSide</i>	[in] 0 = 正面 · 1 = 背面。
<i>nPanel</i>	[in] 0 = YMCO · 1 = K · 2 = Rewrite。

szAreaNameDefault [in] 默认的打印区域名称。

返回值

如果函数执行成功,返回值为 0. 否则返回值为 WIN32 错误代码.

2.19 SOY_PR_DeleteAllJobs

将保存的打印工作全部删除。(只限 Windows 系统)

语法

```
uint32_t __stdcall SOY_PR_DeleteAllJobsW(const WCHAR* szPrinterNameW);  
uint32_t __stdcall SOY_PR_DeleteAllJobsA(const char* szPrinterNameA);
```

参数

szPrinterName [in] 打印机队列名称。

返回值

如果函数执行成功,返回值为 0. 否则返回值为 WIN32 错误代码.

2.20 SOY_PR_StartPrinting2

开始简单打印工作。

语法

```
uint32_t __stdcall SOY_PR_StartPrinting2W(const WCHAR* szPrinterNameW, uint8_t*  
lplnDocProp, HDC* lphPrinterDC);  
uint32_t __stdcall SOY_PR_StartPrinting2A(const char* szPrinterNameA, uint8_t*  
lplnDocProp, HDC* lphPrinterDC);
```

参数

szPrinterName [in]打印机队列名称。

lplnDocProp [in]SEAORY_DOC_PROP 结构的指针, 指定工作要更改的设置.

lphPrinterDC [out] 打印机设备上下文。

返回值

如果函数执行成功,返回值为 0. 否则返回值为 WIN32 错误代码.

2.21 SOY_PR_StartPage2

开始一个打印页面。

语法

```
uint32_t __stdcall SOY_PR_StartPage2(HDC hPrinterDC);
```

参数

hPrinterDC [in] 打印机设备上下文。

返回值

如果函数执行成功,返回值为 0. 否则返回值为 WIN32 错误代码.

2.22 SOY_PR_EndPage2

结束一个打印页面。

语法

```
uint32_t __stdcall SOY_PR_EndPage2(HDC hPrinterDC);
```

参数

hPrinterDC [in] 打印机设备上下文。

返回值

如果函数执行成功,返回值为 0. 否则返回值为 WIN32 错误代码.

2.23 SOY_PR_EndPrinting2

结束简单打印工作。

语法

```
uint32_t __stdcall SOY_PR_EndPrinting2(HDC hPrinterDC, BOOL bCancelJob);
```

参数

hPrinterDC [in] 打印机设备上下文。

bCancelJob [in] 当前面调用的函数返回错误时，将此参数设置为 TRUE，用来取消当前打印工作。

返回值

如果函数执行成功,返回值为 0. 否则返回值为 WIN32 错误代码.

2.24 SOY_PR_PrintImage2

打印图档。

语法

```
uint32_t __stdcall SOY_PR_PrintImage2W(HDC hPrinterDC, int32_t nX, int32_t nY,
int32_t nWidth, int32_t nHeight, const WCHAR* szImageUrlW);
uint32_t __stdcall SOY_PR_PrintImage2A(HDC hPrinterDC, int32_t nX, int32_t nY,
int32_t nWidth, int32_t nHeight, const char* szImageUrlA);
```

参数

<i>hPrinterDC</i>	[in] 打印机设备上下文。
<i>nX</i>	[in] 图像位置 X (pixel)。
<i>nY</i>	[in] 图像位置 Y (pixel)。
<i>nWidth</i>	[in] 图像宽度 (pixel)。
<i>nHeight</i>	[in] 图像高度 (pixel)。若 <i>nWidth</i> 或 <i>nHeight</i> 任一为零则以图档原始宽度及高度打印。
<i>szImageUrl</i>	[in] 图档完整路径。支持 *.jpg, *.bmp, *.png, *.gif。可支持 http://。

返回值

如果函数执行成功，返回值为 0，否则返回值为 WIN32 错误代码。

此函数将检查绘图图像是否在卡的范围之外。如果图形超出卡片范围，则返回值为 0x00011013。

2.25 SOY_PR_PrintText2

打印文字。

语法

```
uint32_t __stdcall SOY_PR_PrintText2W(HDC hPrinterDC, int32_t nX, int32_t nY, const
WCHAR* szTextW, const WCHAR* szFontNameW, int32_t nFontSize, int32_t
nFontWeight, uint8_t byFontAttribute, uint8_t byCharSet, uint32_t dwTextColor, BOOL
bTransparentBack);
uint32_t __stdcall SOY_PR_PrintText2A(HDC hPrinterDC, int32_t nX, int32_t nY, const
char* szTextA, const char* szFontNameA, int32_t nFontSize, int32_t nFontWeight,
uint8_t byFontAttribute, uint8_t byCharSet, uint32_t dwTextColor, BOOL
bTransparentBack);
```

参数

<i>hPrinterDC</i>	[in] 打印机设备上下文。
<i>nX</i>	[in] 文字起始位置 X (pixel)。
<i>nY</i>	[in] 文字起始位置 Y (pixel)。

szText [in] 要打印的文字。

szFontName [in] 文字的字型。

nFontSize [in] 大于零表示文字字号，小于零表示文字高度(单位是像素)。

$lfHeight = -MulDiv(PointSize, GetDeviceCaps(hdc, LOGPIXELSY), 72);$

举例，字号 8.5pt，则 300DPI 下的字高为：

$8.5pt * 300.0 / 72.0 = 35$ 像素

nFontWeight [in] 文字粗体。(值可以为 0, 100, 200, 300, 400, ... 900)

byFontAttribute [in] 旗标 0x40=斜体, 旗标 0x20=下划线。

byCharSet [in] 字元集。参照 LOGFONT.lfCharSet。

dwTextColor [in] 文字颜色。格式为 0x00bbggrr。

bTransparentBack [in] 值为 TRUE 则文字背景透明。

返回值

如果函数执行成功，返回值为 0，否则返回值为 WIN32 错误代码。

此函数将检查文本是否打印在卡的范围之外。如果文本超出卡片范围，则返回值为 0x00011013。

2.26 SOY_PR_PrintOneCard

将一张卡的图像数据发送到打印机内存。

语法

```
uint32_t __stdcall SOY_PR_PrintOneCardW(const WCHAR* szPrinterNameW,
PRINT_CARD_PARAM *lpJobPara);
uint32_t __stdcall SOY_PR_PrintOneCardA(const char* szPrinterNameA,
PRINT_CARD_PARAM *lpJobPara);
```

参数

szPrinterName [in] 打印机队列名称。

lpJobPara [in] 指定打印作业要使用的设置，PRINT_CARD_PARAM 结构的指针。

返回值

如果函数执行成功,返回值为 0. 否则返回值为 WIN32 错误代码.

2.27 SOY_PR_GetBinFirmwareVersion

通过读取固件文件获取固件版本号。

语法

```
uint32_t __stdcall SOY_PR_GetBinFirmwareVersionW(const WCHAR* szBinUrlW,  
WCHAR* szBinVerOutW);  
uint32_t __stdcall SOY_PR_GetBinFirmwareVersionA(const char* szBinUrlA, char*  
szBinVerOutA);
```

参数

szBinUrl [in] 带完整路径的固件文件名。

szBinVerOut [out] 接收 BIN 文件固件版本的缓冲区地址。

返回值

如果函数执行成功,返回值为 0. 否则返回值为 WIN32 错误代码.

2.28 SOY_PR_UpdateFirmware

将固件文件数据发送到打印机以更新固件。

语法

```
uint32_t __stdcall SOY_PR_UpdateFirmwareW(const WCHAR* szPrinterNameW, const  
WCHAR* szBinUrlW);  
uint32_t __stdcall SOY_PR_UpdateFirmwareA(const char* szPrinterNameA, const char*  
szBinUrlA);
```

参数

szPrinterName [in] 打印机队列名称。

szBinUrl [in] 带完整路径的固件文件名。

返回值

如果函数执行成功,返回值为 0. 否则返回值为 WIN32 错误代码.

3. SR30/SR40 智能卡模块函数

3.1 SOY_SC_Init

建立设备的通讯并且分配相应的资源，大部分功能接口都需要在此过程后才能进行。在不需使用设备后，请使用 SOY_SC_Exit 去关闭设备的通讯和释放资源。

语法

```
HANDLE __stdcall SOY_SC_Init(short port);
```

参数

port [in] 0~99 - 表示串口模式（编号物理对应），编号0表示第一个串口合法设备，编号1表示第二个串口合法设备，依此类推。

100~199 - 表示 USB 模式（编号逻辑对应），编号100表示第一个 USB 合法设备，编号101表示第二个 USB 合法设备，依此类推。

返回值

返回值是智能卡模块的通信标识符。

如果函数执行成功，返回值为大于零。否则返回值为 SR30/SR40 错误代码。

3.2 SOY_SC_Exit

关闭设备。

语法

```
short __stdcall SOY_SC_Exit(HANDLE hDev);
```

参数

hDev [in] SOY_SC_Init() 返回的智能卡模块通信标识符。

返回值

如果函数执行成功，返回值为零。否则返回值为 SR30/SR40 错误代码。

3.3 SOY_SC_GetVer

获取设备内部固件代码的版本。

语法

```
short __stdcall SOY_SC_GetVer(HANDLE hDev, unsigned char *sver);
```

参数

hDev [in] SOY_SC_Init() 返回的智能卡模块通信标识符。

sver [out] 返回的版本字符串，请至少分配128个字节。

返回值

如果函数执行成功,返回值为零。否则返回值为 SR30/SR40 错误代码。

3.4 SOY_SC_ResetDevice

将读取器重置为初始化状态。

语法

```
short __stdcall SOY_SC_ResetDevice(HANDLE hDev);
```

参数

hDev[in]SOY_SC_Init() 返回的智能卡模块通信标识符。

返回值

如果函数执行成功,返回值为零。否则返回值为 SR30/SR40 错误代码。

3.5 SOY_SC_Beep

让智能卡模块发出蜂鸣音。

语法

```
short __stdcall SOY_SC_Beep(HANDLE hDev, unsigned short _Msec);
```

参数

hDev [in]SOY_SC_Init() 返回的智能卡模块通信标识符。

_Msec [in] 蜂鸣时间, 单位 10毫秒.

返回值

如果函数执行成功,返回值为零。否则返回值为 SR30/SR40 错误代码。

3.6 SOY_SC_Read_EEPROM

读取设备内部 EEPROM 中的数据，可以用作数据保存等。

语法

```
short __stdcall SOY_SC_Read_EEPROM(HANDLE hDev, short offset, short length,  
unsigned char *rec_buffer);  
short __stdcall SOY_SC_Read_EEPROM_hex(HANDLE hDev, short offset, short length,  
unsigned char *rec_buffer);
```

参数

hDev [in] SOY_SC_Init() 返回的智能卡模块通信标识符。

offset [in] 偏移地址。

length [in] 读取长度。

rec_buffer [out] 返回的数据。

SOY_SC_Read_EEPROM_hex() 为 SOY_SC_Read_EEPROM() 的 HEX 形式接口，参数 *rec_buffer* 为 HEX 格式。

返回值

如果函数执行成功,返回值为零。否则返回值为 SR30/SR40 错误代码。

3.7 SOY_SC_Write_EEPROM

写入数据到设备内部 EEPROM 中，可以用作数据保存等。

语法

```
short __stdcall SOY_SC_Write_EEPROM(HANDLE hDev, short offset, short length,  
unsigned char *send_buffer);  
short __stdcall SOY_SC_Write_EEPROM_hex(HANDLE hDev, short offset, short length,  
unsigned char *send_buffer);
```

参数

hDev [in] SOY_SC_Init() 返回的智能卡模块通信标识符。

offset [in] 偏移地址。

length [in] 写入长度。

send_buffer [in] 传入数据。

SOY_SC_Write_EEPROM_hex() 为 SOY_SC_Write_EEPROM() 的 HEX 形式接口，参数 *send_buffer* 为 HEX 格式。

返回值

如果函数执行成功,返回值为零。否则返回值为 SR30/SR40 错误代码。

3.8 SOY_SC_Load_Key

装载 M1 卡密码到设备内部，装载后可以在特定时候用 SOY_SC_Authentication() 来验证 M1 卡密码。注意：由于多次装载设备密码可能会受硬件存储生命周期限制，此接口只用于密码相对固定，装载次数少的场合。

语法

```
short __stdcall SOY_SC_Load_Key(HANDLE hDev, unsigned char _Mode, unsigned char _SecNr, unsigned char *_NKey);  
short __stdcall SOY_SC_Load_Key_hex(HANDLE hDev, unsigned char _Mode, unsigned char _SecNr, char *_NKey);
```

参数

- hDev* [in] SOY_SC_Init() 返回的智能卡模块通信标识符。
- _Mode* [in] 模式，同 SOY_SC_Authentication 的 _Mode。
- _SecNr* [in] 对应装载的扇区号。
- _NKey* [in] 写入设备中的卡密码，固定为6个字节。

SOY_SC_Load_Key_hex() 为 SOY_SC_Load_Key() 的 HEX 形式接口，参数 _NKey 为 HEX 格式。

返回值

如果函数执行成功,返回值为零。否则返回值为 SR30/SR40 错误代码。

3.9 SOY_SC_Hex_A

字符串变换函数，将十六进制数组转换为文本字符。

语法

```
short __stdcall SOY_SC_Hex_A(unsigned char *hex, unsigned char *a, short len);
```

参数

- hex* [in] 要转换的数据。
- a* [out] 转换后的字符串。
- len* [in] 要转换数据的长度。

返回值

如果函数执行成功,返回值为零。否则返回值为 SR30/SR40 错误代码。

3.10 SOY_SC_A_Hex

字符串变换函数，文本字符转换为十六进制数组。

语法

```
short __stdcall SOY_SC_A_Hex(unsigned char *a, unsigned char *hex, short len);
```

参数

a [in] 要转换的字符串。
hex [out] 转换后的数据。
len [in] 要转换字符串的长度。

返回值

如果函数执行成功,返回值为零。否则返回值为 SR30/SR40 错误代码。

3.11 SOY_SC_Des

单 DES 加密/解密。

语法

```
short __stdcall SOY_SC_Des(unsigned char *key, unsigned char *sour, unsigned char *dest, short m);
```

```
short __stdcall SOY_SC_Des_hex(unsigned char *key, unsigned char *sour, unsigned char *dest, short m);
```

参数

key [in] 8字节密钥。
sour [in] 要加密/解密的8字节数据。
dest [out] 接收8字节结果数据的缓冲区。
m [in] 模式。0是解密, 1 是加密。

SOY_SC_Des_hex () 为 SOY_SC_Des() 的 HEX 形式接口, 参数 key sour dest 为 HEX 格式。

返回值

如果函数执行成功,返回值为零。否则返回值为 SR30/SR40 错误代码。

3.12 SOY_SC_TripleDes

三重 DES 加密/解密。

语法

```
short __stdcall SOY_SC_TripleDes(unsigned char *key, unsigned char *src, unsigned char *dest, short m);
```

```
short __stdcall SOY_SC_TripleDes_hex(unsigned char *key, unsigned char *src, unsigned char *dest, short m);
```

参数

- key* [in]8字节密钥。
- src* [in]要加密/解密的8字节数据。
- dest* [out]接收8字节结果数据的缓冲区。
- m* [in]模式。0是解密, 1 是加密。

SOY_SC_TripleDes_hex () 为 SOY_SC_TripleDes() 的 HEX 形式接口, 参数 *key* *src* *dest* 为 HEX 格式。

返回值

如果函数执行成功,返回值为零。否则返回值为 SR30/SR40 错误代码。

3.13 SOY_SC_Set_CPU

设置当前接触式卡座为指定卡座, 用于多卡座切换卡操作。

语法

```
short __stdcall SOY_SC_Set_CPU(HANDLE hDev, unsigned char _Byte);
```

参数

- hDev* [in]SOY_SC_Init() 返回的智能卡模块通信标识符。
- _Byte* [in] 卡座编号。
 - 0x0C - 附卡座/接触式 CPU1卡座。
 - 0x0B - 接触式 CPU2卡座。
 - 0x0D - SAM1卡座。
 - 0x0E - SAM2卡座。
 - 0x0F - SAM3卡座。
 - 0x11 - SAM4卡座。

返回值

如果函数执行成功,返回值为零。否则返回值为 SR30/SR40 错误代码。

3.14 SOY_SC_CpuReset

CPU 卡的通电和复位功能, 可以在复位后自动判断卡的协议。

语法

```
short __stdcall SOY_SC_CpuReset(HANDLE hDev, unsigned char *rlen, unsigned char *databuffer);
```

```
short __stdcall SOY_SC_CpuReset_hex(HANDLE hDev, unsigned char *rlen, char *databuffer);
```

参数

hDev [in]SOY_SC_Init() 返回的智能卡模块通信标识符。

rlen [in]返回复位信息的长度。

databuffer [out]返回的复位信息，请至少分配128个字节。

SOY_SC_CpuReset_hex() 为 SOY_SC_CpuReset 的 HEX 形式接口，参数 databuffer 为 HEX 格式。

返回值

如果函数执行成功,返回值为零。否则返回值为 SR30/SR40 错误代码。

3.15 SOY_SC_Reset

射频复位功能智能卡模块。

语法

```
short __stdcall SOY_SC_Reset(HANDLE hDev, unsigned short nMsec);
```

参数

hDev [in]SOY_SC_Init() 返回的智能卡模块通信标识符。

nMsec [in]重置的时间, 单位 milli-second. 0 表示要关闭无线电频率.

返回值

如果函数执行成功,返回值为零。否则返回值为 SR30/SR40 错误代码。

3.16 SOY_SC_ConfigCard

设置读取器要操作的非接触卡类型. 读取器开机后, 默认状态是操作 A 类型卡。

语法

```
short __stdcall SOY_SC_ConfigCard(HANDLE hDev, unsigned char cardtype);
```

参数

hDev [in]SOY_SC_Init() 返回的智能卡模块通信标识符。

cardtype [in] 'A' 表示 Type A 卡, 'B' 表示 Type B 卡。

返回值

如果函数执行成功,返回值为零。否则返回值为 SR30/SR40 错误代码。

3.17 SOY_SC_Card_n

寻卡请求、防卡冲突、选卡操作。

语法

```
short __stdcall SOY_SC_Card_n(HANDLE hDev, unsigned char _Mode, unsigned int *SnrLen, unsigned char *_Snr);
```

参数

- hDev* [in] SOY_SC_Init() 返回的智能卡模块通信标识符。
- _Mode* [in] 寻找卡模式。_Mode=0: 空闲模式。_Mode=1: 所有模式。
- SnrLen* [out] 返回卡序列号的长度。
- _Snr* [out] 缓冲区的地址, 以接收返回的卡序列号。

返回值

如果函数执行成功,返回值为零。否则返回值为 SR30/SR40 错误代码。

3.18 SOY_SC_AuthenticationPassAddr

验证 M1 卡密码。

语法

```
short __stdcall SOY_SC_AuthenticationPassAddr(HANDLE hDev, unsigned char _Mode, unsigned char _Addr, unsigned char *passbuff);
```

参数

- hDev* [in] SOY_SC_Init() 返回的智能卡模块通信标识符。
- _Mode* [in] 模式。0x00表示验证 A 密码, 0x04表示验证 B 密码。
- _Addr* [in] 要验证密码的扇区号。
- passbuff* [in] 密码, 固定为6个字节。

返回值

如果函数执行成功,返回值为零。否则返回值为 SR30/SR40 错误代码。

3.19 SOY_SC_Write

写入数据到卡片内, 对于 M1 卡, 一次必须写入一个块的数据, 为 16 个字节; 对于 ML 卡, 一次必须写入一个页的数据, 为 4 个字节。

语法

```
short __stdcall SOY_SC_Write(HANDLE hDev, unsigned char _Adr, unsigned char *_Data);
```

参数

- hDev* [in] SOY_SC_Init() 返回的智能卡模块通信标识符。
- _Adr* [in] M1卡 - S50块地址 (1~63) , S70块地址 (1~255) 。 ML 卡 - 页地址 (2~11) 。
- _Data* [in] 固定传入16个字节数据 , 真实数据可能小于16个字节。

返回值

如果函数执行成功,返回值为零。否则返回值为 SR30/SR40 错误代码。

3.20 SOY_SC_Read

读取卡内数据 , 对于 M1 卡 , 一次读取一个块的数据 , 为 16 个字节 ; 对于 ML 卡 , 一次读取相同属性的两页 , 为 8 个字节。

语法

```
short __stdcall SOY_SC_Read(HANDLE hDev, unsigned char _Adr, unsigned char *_Data);
```

参数

- hDev* [in] SOY_SC_Init() 返回的智能卡模块通信标识符。
- _Adr* [in] M1卡 - S50块地址 (0~63) , S70块地址 (0~255) 。 ML 卡 - 页地址 (0~11) 。
- _Data* [out] 固定返回16个字节数据 , 真实数据可能小于16个字节。

返回值

如果函数执行成功,返回值为零。否则返回值为 SR30/SR40 错误代码。

3.21 SOY_SC_InitVal

用于操作 M1 卡 , 使得指定块为值块 , 并且初始化此值块为指定数值。

语法

```
short __stdcall SOY_SC_InitVal(HANDLE hDev, unsigned char _Adr, unsigned int _Value);
```

参数

- hDev* [in] SOY_SC_Init() 返回的智能卡模块通信标识符。

_Adr [in] 块地址。

_Data [in] 初始化数值。

返回值

如果函数执行成功,返回值为零。否则返回值为 SR30/SR40 错误代码。

3.22 SOY_SC_ReadVal

用于读取 M1 卡值块数值。

语法

```
short __stdcall SOY_SC_ReadVal(HANDLE hDev, unsigned char _Adr, unsigned int *_Value);
```

参数

hDev [in]SOY_SC_Init() 返回的智能卡模块通信标识符。

_Adr [in] 块地址。

_Value [out] 返回的数值。

返回值

如果函数执行成功,返回值为零。否则返回值为 SR30/SR40 错误代码。

3.23 SOY_SC_Increment

用于操作 M1 卡值块,使块值增加指定数值。

语法

```
short __stdcall SOY_SC_Increment(HANDLE hDev, unsigned char _Adr, unsigned int *_Value);
```

参数

hDev [in]SOY_SC_Init() 返回的智能卡模块通信标识符。

_Adr [in] 块地址。

_Value [in] 要增加的数值。

返回值

如果函数执行成功,返回值为零。否则返回值为 SR30/SR40 错误代码。

3.24 SOY_SC_Decrement

用于操作 M1 卡值块，使块值减少指定数值。

语法

```
short __stdcall SOY_SC_Decrement(HANDLE hDev, unsigned char _Adr, unsigned int _Value);
```

参数

hDev [in]SOY_SC_Init() 返回的智能卡模块通信标识符。

_Adr [in] 块地址。

_Value [in] 要减少的数值。

返回值

如果函数执行成功,返回值为零。否则返回值为 SR30/SR40 错误代码。

3.25 SOY_SC_Restore

用于把指定的值块内容暂存，后续可以使用 SOY_SC_Transfer 将暂存值块内容传递到另一块中，实现块与块之间数值传送。

语法

```
short __stdcall SOY_SC_Restore(HANDLE hDev, unsigned char _Adr);
```

参数

hDev [in]SOY_SC_Init() 返回的智能卡模块通信标识符。

_Adr [in] 要回传的块地址。

返回值

如果函数执行成功,返回值为零。否则返回值为 SR30/SR40 错误代码。

3.26 SOY_SC_Transfer

用于把 SOY_SC_Restore 暂存的值块内容传递到指定块中，实现块与块之间数值传送。

语法

```
short __stdcall SOY_SC_Transfer(HANDLE hDev, unsigned char _Adr);
```

参数

hDev [in]SOY_SC_Init() 返回的智能卡模块通信标识符。

_Adr [in] 要传递的块地址。

返回值

如果函数执行成功,返回值为零。否则返回值为 SR30/SR40 错误代码。

3.27 SOY_SC_Pro_ResetInt

对感应区 CPU 卡进行复位操作。

语法

```
short __stdcall SOY_SC_Pro_ResetInt(HANDLE hDev, unsigned char *rlen, unsigned char *receive_data);
```

参数

- hDev* [in]SOY_SC_Init() 返回的智能卡模块通信标识符。
- rlen* [out] 返回复位信息的长度。
- receive_data* [out] 返回的复位信息，请至少分配128个字节。

返回值

如果函数执行成功,返回值为零。否则返回值为 SR30/SR40 错误代码。

3.28 SOY_SC_Pro_CommandLinkInt

APDU 的信息交换功能, 该功能已封装了 T = CL 操作。

语法

```
short __stdcall SOY_SC_Pro_CommandLinkInt(HANDLE hDev, unsigned int slen, unsigned char *sendbuffer, unsigned int *rlen, unsigned char *databuffer, unsigned char timeout);
```

参数

- hDev* [in]SOY_SC_Init() 返回的智能卡模块通信标识符。
- slen* [in] 发送数据的长度。
- sendbuffer* [in] 发送数据。
- rlen* [out] 返回数据的长度。
- databuffer* [out] 返回的数据。
- timeout* [in]超时值。单位是250ms。通常设置为7。

返回值

如果函数执行成功,返回值为零。否则返回值为 SR30/SR40 错误代码。

3.29 SOY_SC_Card_B

对 Type B 卡进行寻卡和激活。

语法

```
short __stdcall SOY_SC_Card_B(HANDLE hDev, unsigned char *rbuf);
```

参数

hDev [in]SOY_SC_Init() 返回的智能卡模块通信标识符。

rbuf [out] 返回的激活信息，请至少分配128个字节。

返回值

如果函数执行成功,返回值为零。否则返回值为 SR30/SR40 错误代码。

3.30 SOY_SC_Pro_Reset

对感应区 CPU 卡进行复位操作。

语法

```
short __stdcall SOY_SC_Pro_Reset(HANDLE hDev, unsigned char *rlen, unsigned char *receive_data);
```

参数

hDev [in]SOY_SC_Init() 返回的智能卡模块通信标识符。

rlen [out] 返回复位信息的长度。

receive_data [out] 返回的复位信息，请至少分配128个字节。

返回值

如果函数执行成功,返回值为零。否则返回值为 SR30/SR40 错误代码。

3.31 SOY_SC_MF_PLUS_WritePerson

设置 Mifare Plus 卡的个人化数据。

语法

```
short __stdcall SOY_SC_MF_PLUS_WritePerson(HANDLE hDev, unsigned int BNr, unsigned char *dataperso);
```

参数

hDev [in]SOY_SC_Init() 返回的智能卡模块通信标识符。
BNr [in]要写入的块号。
dataperso [in]要写入的数据。16字节。

返回值

如果函数执行成功,返回值为零。否则返回值为 SR30/SR40 错误代码。

3.32 SOY_SC_Auth_Ulc

使用传入的密码来验证 Mifare Ultralight C 卡密码。

语法

```
short __stdcall SOY_SC_Auth_Ulc(HANDLE hDev, unsigned char *key);
```

参数

hDev [in]SOY_SC_Init() 返回的智能卡模块通信标识符。
key [in] 密码，固定为16个字节。

返回值

如果函数执行成功,返回值为零。否则返回值为 SR30/SR40 错误代码。

3.33 SOY_SC_VerifyPin_4442

使用传入的密码来验证 4442 卡密码。

语法

```
short __stdcall SOY_SC_VerifyPin_4442(HANDLE hDev, unsigned char *passwd);
```

参数

hDev [in]SOY_SC_Init() 返回的智能卡模块通信标识符。
passwd [in]要验证的3字节密码。

返回值

如果函数执行成功,返回值为零。否则返回值为 SR30/SR40 错误代码。

3.34 SOY_SC_Write_4442

将数据写入 4442 卡。

语法

```
short __stdcall SOY_SC_Write_4442(HANDLE hDev, short offset, short length,  
unsigned char *data_buffer);
```

参数

hDev [in]SOY_SC_Init() 返回的智能卡模块通信标识符。

offset [in] 要写入数据的偏移地址。

length [in] 要写入的数据的长度。

data_buffer[in] 要写入的数据。

返回值

如果函数执行成功,返回值为零。否则返回值为 SR30/SR40 错误代码。

3.35 SOY_SC_Read_4442

从 4442 读取数据卡。

语法

```
short __stdcall SOY_SC_Read_4442(HANDLE hDev, short offset, short length, unsigned  
char *data_buffer);
```

参数

hDev [in]SOY_SC_Init() 返回的智能卡模块通信标识符。

offset [in]要读取数据的偏移地址。

length [in]要读取的数据的长度。

data_buffer[out]从卡接收数据的缓冲区。

返回值

如果函数执行成功,返回值为零。否则返回值为 SR30/SR40 错误代码。

3.36 SOY_SC_VerifyPin_4428

验证 4428 卡密码。

语法

```
short __stdcall SOY_SC_VerifyPin_4428(HANDLE hDev, unsigned char *passwd);
```

参数

hDev [in]SOY_SC_Init() 返回的智能卡模块通信标识符。
passwd[in]要验证的2字节密码。

返回值

如果函数执行成功,返回值为零。否则返回值为 SR30/SR40 错误代码。

3.37 SOY_SC_Write_4428

将数据写入 4428 卡.

语法

```
short __stdcall SOY_SC_Write_4428(HANDLE hDev, short offset, short length,  
unsigned char *data_buffer);
```

参数

hDev [in]SOY_SC_Init() 返回的智能卡模块通信标识符。
offset [in]要写入数据的偏移地址。
length [in]要写入的数据的长度。
Data_buffer[in]要写入的数据。

返回值

如果函数执行成功,返回值为零。否则返回值为 SR30/SR40 错误代码。

3.38 SOY_SC_Read_4428

从 4428 读取数据卡.

语法

```
short __stdcall SOY_SC_Read_4428(HANDLE hDev, short offset, short length, unsigned  
char *data_buffer);
```

参数

hDev [in]SOY_SC_Init() 返回的智能卡模块通信标识符。
offset [in]要读取数据的偏移地址。
length [in]要读取的数据的长度。
data_buffer[out]从卡接收数据的缓冲区。

返回值

如果函数执行成功,返回值为零。否则返回值为 SR30/SR40 错误代码。

3.39 SOY_SC_Write_24c

将数据写入 24Cxx 卡。

语法

```
short __stdcall SOY_SC_Write_24c(HANDLE hDev, short offset, short length, unsigned char *snd_buffer);
```

参数

- hDev* [in]SOY_SC_Init() 返回的智能卡模块通信标识符。
- offset* [in]要写入数据的偏移地址。
- length* [in]要写入的数据的长度。
- snd_buffer* [in]要写入的数据。

返回值

如果函数执行成功,返回值为零。否则返回值为 SR30/SR40 错误代码。

3.40 SOY_SC_Read_24c

从 24Cxx 卡读取数据。

语法

```
short __stdcall SOY_SC_Read_24c(HANDLE hDev, short offset, short length, unsigned char *receive_buffer);
```

参数

- hDev* [in]SOY_SC_Init() 返回的智能卡模块通信标识符。
- offset* [in]要读取数据的偏移地址。
- length* [in]要读取的数据的长度。
- Receive_buffer* [out]从卡接收数据的缓冲区。

返回值

如果函数执行成功,返回值为零。否则返回值为 SR30/SR40 错误代码。

3.41 SOY_SC_CpuApduInt

CPU 卡 APDU 的信息交换功能. 他的功能封装了 T=0 和 T=1 操作。

语法

```
short __stdcall SOY_SC_CpuApduInt(HANDLE hDev, unsigned int slen, unsigned char *sendbuffer, unsigned int *rlen, unsigned char *databuffer);
```

参数

hDev [in] SOY_SC_Init() 返回的智能卡模块通信标识符。

slen [in] 发送数据的长度。

sendbuffer [in] 发送数据。

rlen [out] 返回数据的长度。

databuffer [out] 返回的数据。

返回值

如果函数执行成功,返回值为零。否则返回值为 SR30/SR40 错误代码。

3.42 SOY_SC_Card

请求卡并返回卡的序列号。

语法

```
short __stdcall SOY_SC_Card(HANDLE hDev, unsigned char _Mode, unsigned int *_Snr);
```

参数

hDev [in] SOY_SC_Init() 返回的智能卡模块通信标识符。

_Mode [in] 寻卡模式. Mode=0: 空闲模式. Mode=1: 所有模式.

_Snr [out] 返回的卡序列号。

返回值

如果函数执行成功,返回值为零。否则返回值为 SR30/SR40 错误代码。

3.43 SOY_SC_CpuApdu

CPU 卡 APDU 的信息交换功能. 他的功能封装了 T=0 和 T=1 操作。

语法

```
short __stdcall SOY_SC_CpuApdu(HANDLE hDev, unsigned char slen, unsigned char *sendbuffer, unsigned char *rlen, unsigned char *databuffer);
```

参数

hDev [in] SOY_SC_Init() 返回的智能卡模块通信标识符。
slen [in] 发送数据的长度。
sendbuffer [in] 发送数据。
rlen [out] 返回数据的长度。
databuffer [out] 返回的数据。

返回值

如果函数执行成功,返回值为零。否则返回值为 SR30/SR40 错误代码。

3.44 SOY_SC_Pro_CommandLink

APDU 的信息交换功能, 该功能已封装了 T = CL 操作。

语法

```
short __stdcall SOY_SC_Pro_CommandLink(HANDLE hDev, unsigned char slen, unsigned char *sendbuffer, unsigned char *rlen, unsigned char *databuffer, unsigned char timeout, unsigned char FG);
```

参数

hDev [in] SOY_SC_Init() 返回的智能卡模块通信标识符。
slen [in] 发送数据的长度。
sendbuffer [in] 发送数据。
rlen [out] 返回数据的长度。
databuffer [out] 返回的数据。
timeout [in] 超时值。单位是250ms。通常设置为7。
FG [in] 分割值。建议此值为64。

返回值

如果函数执行成功,返回值为零。否则返回值为 SR30/SR40 错误代码。

4. UHF 模块函数

4.1 UhfReaderConnect

打开端口并连接.

语法

```
int WINAPI UhfReaderConnect(HANDLE &hCom, const char* cPort, UCHAR flagCrc);
```

参数

hCom [out]返回打开的 COM 端口句柄。

cPort [in]COM 端口名称。如 COM1、COM2、COM3 等。

flagCrc [in]波特率。0意味着波特9600, 2 意味着波特19200, 4 意味着波特57600, 6 意味着波特115200。

返回值

如果函数执行成功,返回值为 大于零。

4.2 UhfReaderDisconnect

断开和关闭端口.

语法

```
int WINAPI UhfReaderDisconnect (HANDLE &hCom, UCHAR flagCrc);
```

参数

hCom [in]要关闭的 COM 端口句柄。

flagCrc [in]设置为零。

返回值

如果函数执行成功,返回值为大于零。

4.3 UhfGetPower

获取 UHF 模块电源设置.

语法

```
int WINAPI UhfGetPower (HANDLE hCom, char* uPower, UCHAR flagCrc);
```

参数

hCom [in] UhfReaderConnect() 返回的 COM 端口句柄。

uPower [out]接收电源设置字符串的缓冲区。

flagCrc [in]设置为零。

返回值

如果函数执行成功,返回值为大于零。

4.4 UhfSetPower

UHF 模块电源设置.

语法

```
int WINAPI UhfSetPower (HANDLE hCom, char uOption, int uPower, UCHAR flagCrc);
```

参数

hCom [in] UhfReaderConnect() 返回的 COM 端口句柄。

uOption [in]设置为零。

uPower [in]P 功率要设置的值.

flagCrc [in]设置为零。

返回值

如果函数执行成功,返回值为大于零。

4.5 UhfSetFrequency

UHF 模块 频率 设置.

语法

```
int WINAPI UhfSetFrequency(HANDLE hCom, UCHAR uFreMode, UCHAR uFreBase,
UCHAR* uBaseFre, UCHAR uChannNum, UCHAR uChannSpc, UCHAR uFreHop,
UCHAR flagCrc);
```

参数

hCom [in] UhfReaderConnect() 返回的 COM 端口句柄。

uFreMode [in]设置为零。

uFreBase [in]设置为零。

uBaseFre [in]设置为零。

uChannNum [in]01 = 中国标准 920 ~ 925 m。
02 = 美国标准 902 ~ 928 m。

03 = 欧洲标准 865 ~ 868 m。

04 = 中国标准 840 ~ 845 m。

06 = 韩国标准 917 ~ 923 m。

uChannSpc [in] 设置为零。

uFreHop [in] 设置为零。

flagCrc [in] 设置为零。

返回值

如果函数执行成功,返回值为大于零。

4.6 UhfReadDataByEPC

读数据从超高频卡。

语法

```
int WINAPI UhfReadDataByEPC (HANDLE hCom, const char* accessPwd, int memBank, int sa, int dl, UCHAR* uDataReturn, UCHAR flagCrc);
```

参数

hCom [in] UhfReaderConnect() 返回的 COM 端口句柄。

accessPwd [in] 字符串4字节十六进制值作为密码。

memBank [in] 数据区。0 = 预留, 1 = EPC, 2 = TID, 3 = 用户。

sa [in] 设置为零。

dl [in] 设置为0x12。

uDataReturn [out] 接收返回数据的缓冲区。

flagCrc [in] 设置为零。

返回值

如果函数执行成功,返回值为 返回的数据的长度。

4.7 UhfReadDataByTID

读 TID 数据从超高频卡。

语法

```
int WINAPI UhfReadDataByTID (HANDLE hCom, int sa, int dl, UCHAR* uDataReturn, UCHAR* uErrorCode, UCHAR flagCrc);
```

参数

hCom [in] UhfReaderConnect() 返回的 COM 端口句柄。
sa [in] 设置为零。
dl [in] 设置为0x12。
uDataReturn [out] 接收返回数据的缓冲区。
uErrorCode [out] 接收错误代码的缓冲区。
flagCrc [in] 设置为零。

返回值

如果函数执行成功,返回值为返回的数据的长度。

4.8 UhflInventorySingleTag

找单张卡。

语法

```
int WINAPI UhflInventorySingleTag (HANDLE hCom, UCHAR* uLenUii, UCHAR* uUii ,  
UCHAR flagCrc);
```

参数

hCom [in] UhfReaderConnect() 返回的 COM 端口句柄。
uLenUii [out] 接收返回数据长度的缓冲区。
uUii [out] 接收返回数据的缓冲区。
flagCrc [in] 设置为零。

返回值

如果函数执行成功,返回值为大于零。

4.9 UhfAddFilter

添加选择。

语法

```
int WINAPI UhfAddFilter (HANDLE hCom, int intSelTarget, int intAction, int  
intSelMemBank, int intSelPointer, int intMaskLen, int intTruncate, const char*  
txtSelMask, UCHAR flagCrc);
```

参数

hCom [in] UhfReaderConnect() 返回的 COM 端口句柄。
intSelTarget [in] 设置为零。
intAction [in] 设置为零。

intSelMemBank[in] 设置为1。
intSelPointer [in] 设置为32。
intSelMaskLength [in] 设置为32。
intTruncate [in] 设置为零。
txtSelMask [in] EPC 字符串。
flagCrc [in] 设置为零。

返回值

如果函数执行成功,返回值为大于零。

4.10 UhfWriteDataByEPC

写数据到超高频卡。

语法

```
int WINAPI UhfWriteDataByEPC(HANDLE hCom, const char* uAccessPwd, UCHAR  
uBank, const char* uPtr, UCHAR uCnt, UCHAR* uWriteData, UCHAR* uErrorCode,  
UCHAR flagCrc);
```

参数

hCom [in] UhfReaderConnect() 返回的 COM 端口句柄。
accessPwd [in] 字符串4字节十六进制值作为密码。
uBank [in] 数据部分。0 = 预留, 1 = EPC, 2 = 工贸, 3 = 用户。
uPtr [in] 设置为"00 00".
uCnt [in] 按单词计数的数据长度.字节长度/2。
uWriteData [in] 缓冲区的数据写。
uErrorCode [out] 接收错误代码的缓冲区。
flagCrc [in] 设置为零。

返回值

如果函数执行成功,返回值为大于零。

5. 结构

5.1 SEAORY_DOC_PROP

SEAORY_DOC_PROP 结构指定了一般可更改的打印机设置。

```
typedef struct tagSEAORY_DOC_PROP
```

```
{  
    uint8_t      byOrientation;  
    uint8_t      byRibbonType;  
    uint8_t      byPrintSide;  
    uint8_t      byPrintPanelFront;  
    uint8_t      byPrintPanelBack;  
    uint8_t      byResinKfront;  
    uint8_t      byResinKback;  
    uint8_t      byRotate180;  
  
    uint8_t      byInputBin;  
    uint8_t      byFeedCardMode;  
    uint8_t      byOutputBin;  
    uint8_t      byEjectCardMode;  
    uint8_t      byCardInOutByDev;  
    uint8_t      byResolution;  
    uint8_t      byEraseCard;  
  
    uint8_t      byAutoDetectRibbon;  
    uint8_t      byMirror;  
    uint8_t      byMonoSpeedMode;  
  
    uint8_t      byGrayYMC;  
    uint8_t      byPaperSize;  
  
    uint8_t      byReserved[236];  
}
```

```
} SEAORY_DOC_PROP;
```

结构成员描述:

- *byOrientation*
指定打印工作的输出方向。
1 = 纵向 · 2 = 横向。
- *byRibbonType*
指定打印工作使用的色带。
部份机种不使用此栏位: S20R, S22R, S22J。
S 系列: 0=YMCKO, 1=K, 2=1/2ymcKO, 3=YMCKOK, 4=KO, 5=Gold, 6=Silver, 7=White, 8=1/2ymcKOKO, 9=1/2ymcKO-n, 10=RYMCK, 11=UV, 12=Red, 13=Blue。
R 系列: 100=YMCK, 101=YMCKK, 1=K。
- *byPrintSide*
指定工作的打印面。
部份机种不使用此栏位: S20R, S22R。
0x00 = 不打印, 0x01 = 正面 · 0x10 = 背面 · 0x11 = 双面。

- *byPrintPanelFront*
指定正面使用的色带色层。
当色带含有 YMCKO 时 · 0=YMCKO, 1=YMCO, 2=K, 3=KO, 4=YMCK, 5=YMC。
当色带是 RYMCK 时 · 0=RYMCK, 1=RYMC, 2=K, 3=RK, 4=RYMCK+R, 5=YMCK+R。
当色带是 YMCK 时 · 0=YMCK, 1=YMC, 2=K。
- *byPrintPanelBack*
指定背面使用的色带色层。
当色带含有 YMCKO 时 · 0=YMCKO, 1=YMCO, 2=K, 3=KO, 4=YMCK, 5=YMC。
当色带是 RYMCK 时 · 0=RYMCK, 1=RYMC, 2=K, 3=RK, 4=RYMCK+R, 5=YMCK+R。
当色带是 YMCK 时 · 0=YMCK, 1=YMC, 2=K。
- *byResinKfront*
指定正面的 K 色带打印方法。
当色带是 K · Gold · Silver · White 或 KO 时 · 0=递色, 1=黑/白。
彩色带时 · 0=黑色文字, 图像, 图形 · 1=黑色图像 · 2=黑色图形。
- *byResinKback*
指定背面的 K 色带打印方法。
当色带是 K · Gold · Silver · White 或 KO 时 · 0=递色, 1=黑/白。
彩色带时 · 0=黑色文字, 图像, 图形 · 1=黑色图像 · 2=黑色图形。
- *byRotate180*
指定哪一面需要旋转 180 度。
0x00=都不旋转, 0x01=正面旋转, 0x10=背面旋转, 0x11=双面旋转。
- *byInputBin*
指定馈送卡片的来源。
0 =自动进卡器 · 1 =前侧手动插槽 · 2 =后侧手动插槽。
- *byFeedCardMode*
指定是否使用勾卡模式来馈送卡。
0x00 =无勾卡模式, 0x01 =勾卡模式, 0x02 =以勾卡模式重试进卡。
- *byOutputBin*
指定弹出卡的目标。
0 =出卡盒 · 1 =前侧手动插槽 · 2 =后侧手动插槽 · 3 =废卡盒 · 4 =不要排出卡片。
- *byEjectCardMode*
指定弹出卡方法。
0 =直接输出 · 1 =等待取走卡片。
- *byCardInOutByDev*
指定使用的进卡和出卡设置。
0=使用上述 4 个栏位的进卡和出卡设置。
1=使用保存在设备中的进卡和出卡设置。以上 4 个栏位将被忽略。
- *byResolution*
指定打印分辨率。
0=300x300dpi · 1=300x600dpi。
R 系列: 忽略此栏位。
- *byEraseCard*
支持 S20R。指定擦除卡片方式。
0x00=不擦除卡片, 0x01=打印前擦除, 0x80=只擦除不打印。
R 系列: 忽略此栏位。
- *byAutoDetectRibbon*
指定在设备打印之前检测色带类型并应用默认设置。
部份机种不使用此栏位: S20R, S22R, S22J。
0=以 byRibbonType 设置要使用的色带 · 1 =自动检测色带。
- *byMirror*
指定哪一面需要镜像打印。
0x00=都不镜像, 0x01=正面镜像, 0x10=背面镜像, 0x11 =双面镜像。
- *byMonoSpeedMode*

指定在使用树脂 K 色带时使用速度模式。

部份机种不使用此栏位: S20R, S22R, S22J, R 系列。

0 = 正常速度 · 1 = 按速度模式打印单色碳带。

- *byGrayYMC*

指定在通过 YMC 色带打印图像时将彩色图像转换为灰度模式。

0=通过 YMC 进行常规彩色打印 · 1=通过灰度模式打印彩色像素。

- *byPaperSize*

指定打印卡尺寸。

部份机种不使用此栏位: S20R, S22R, S22J, R 系列。

0= CR-80 · 1= 54mm X 114mm (限用单色碳带) · 2= A4 (需要特殊版本驱动程序)。

5.2 PRINT_CARD_PARAM

PRINT_CARD_PARAM 结构指定了打印作业的设置。

typedef struct tagPRINT_CARD_PARAM

```
{
    uint32_t          dwSize;
    uint32_t          dwReserve2;
    uint32_t          dwReserve3;
    uint32_t          dwReserve4;

    uint8_t           byReserve1;
    uint8_t           byReserve2;
    uint8_t           byReserve3;
    uint8_t           byReserve4;
    uint8_t           byReserve5;
    uint8_t           byReserve6;
    uint8_t           byReserve7;
    uint8_t           byReserve8;

    BITMAP            *lpFrontBGR;
    BITMAP            *lpFrontK;
    BITMAP            *lpFrontO;

    BITMAP            *lpBackBGR;
    BITMAP            *lpBackK;
    BITMAP            *lpBackO;

    BITMAP            *lpFrontErase;
    uint8_t           *lpReserve2;

    short             ColorAdj[8];

    char              chFrontHeatYMC;
    char              chFrontHeatK;
    char              chFrontHeatO;
    char              chFrontHeatResinK;

    char              chBackHeatYMC;
    char              chBackHeatK;
```

char	chBackHeatO;
char	chBackHeatResinK;
char	chFrontHeatWrite;
char	chFrontHeatErase;
char	chFrontHeatRcv;
char	chBackHeatRcv;
char	chReserved[172];

} PRINT_CARD_PARAM;

结构成员描述:

- *dwSize*
指定结构的大小，以字节计算。
- *lpFrontBGR*
指定正面彩色的打印数据，为 BITMAP 结构的指针。
以横向来看，宽 x 高如下：(单位像素，深度 24 位)
S 系列: 1012x648。R300: 1030x648。R600/R600M: 2060x1296。
- *lpFrontK*
指定正面黑色的打印数据，为 BITMAP 结构的指针。
- *lpFrontO*
指定正面覆膜层的打印数据，为 BITMAP 结构的指针。
- *lpBackBGR*
指定背面彩色的打印数据，为 BITMAP 结构的指针。
- *lpBackK*
指定背面黑色的打印数据，为 BITMAP 结构的指针。
- *lpBackO*
指定背面覆膜层的打印数据，为 BITMAP 结构的指针。
- *lpFrontErase*
指定要抹除的打印数据，为 BITMAP 结构的指针。
- *ColorAdj*
指定色彩的调整幅度。
ColorAdj[0] = 亮度，数值范围 -100 ~ 100。
ColorAdj[1] = 对比，数值范围 -100 ~ 100。
ColorAdj[2] = 锐利度，数值范围 0 ~ 100。
ColorAdj[3] = 黄色，数值范围 -100 ~ 100。
ColorAdj[4] = 洋红色，数值范围 -100 ~ 100。
ColorAdj[5] = 靛青色，数值范围 -100 ~ 100。
ColorAdj[6] = 饱和度，数值范围 -100 ~ 100。
ColorAdj[7] = 伽玛 (x0.01)，数值范围 10 ~ 999。
- *chFrontHeatYMC*
指定正面的 YMC 加热能量调整值，数值范围 -127 ~ 127。
- *chFrontHeatK*
指定正面的 YMCKO 色带的 K 加热能量调整值，数值范围 -127 ~ 127。
- *chFrontHeatO*
指定正面的覆膜层加热能量调整值，数值范围 -127 ~ 127。
- *chFrontHeatResinK*
指定正面的 K 色带加热能量调整值，数值范围 -127 ~ 127。
- *chBackHeatYMC*
指定背面的 YMC 加热能量调整值，数值范围 -127 ~ 127。
- *chBackHeatK*
指定背面的 YMCKO 色带的 K 加热能量调整值，数值范围 -127 ~ 127。
- *chBackHeatO*

指定背面的覆膜层加热能量调整值，数值范围 -127 ~ 127。

- *chBackHeatResink*

指定背面的 K 色带加热能量调整值，数值范围 -127 ~ 127。

- *chFrontHeatWrite*

指定正面的可复写卡打印的加热能量调整值，数值范围 -127 ~ 127。

- *chFrontHeatErase*

指定正面的可复写卡抹除的加热能量调整值，数值范围 -127 ~ 127。

- *chFrontHeatRcv*

指定正面的 RYMCK 色带的 R 层加热能量调整值，数值范围 -127 ~ 127。

- *chBackHeatRcv*

指定背面的 RYMCK 色带的 R 层加热能量调整值，数值范围 -127 ~ 127。

6. 常量

6.1 打印机错误代码

Printer 错误代码	错误代码的描述
0x00010001	固件错误
0x00010002	加热头热敏电阻错误
0x00010003	编码器错误 (带动端)
0x00010004	编码器错误 (供应端)
0x00010005	ADC 错误
0x00010006	编码器错误 (转印膜) (06)
0x00010007	加热器热敏电阻错误 (07)
0x00010008	FPGA 错误 (08)
0x00010009	烧录 FPGA 错误 (09)
0x0001000A	烧录 LCD 错误 (0A)
0x00010010	夹卡 (10)
0x00010011	夹卡 (11)
0x00010012	夹卡 (12)
0x00010013	夹卡 (13)
0x00010014	夹卡 (14)
0x00010015	夹卡 (15)
0x00010016	夹卡 (16)
0x00010017	夹卡 (17)
0x00010018	夹卡 (18)
0x00010019	夹卡 (19)
0x0001001A	夹卡 (1A)
0x0001001B	夹卡 (1B)
0x00010021	色带门打开
0x00010022	废卡盒打开
0x00010023	废卡盒已满
0x00010024	整平盖打开 (24)
0x00010031	打印头凸轮错误
0x00010032	自动进卡器凸轮错误
0x00010033	翻转器错误
0x00010034	加热器凸轮错误 (34)
0x00010035	整平凸轮错误 (35)
0x00010041	色带用完 (41)
0x00010042	色带错误 (42)
0x00010043	没有放色带 (43)
0x00010044	色带不可用 (44)
0x00010045	没有放色带 (45)
0x00010046	色带用完 (46)
0x00010047	色带与芯片不匹配 (47)
0x00010048	色带错误 (48)
0x00010049	色带错误 (49)
0x0001004A	色带安装错误 (4A)
0x00010051	进卡错误 (51)
0x00010052	进卡错误 (52)

0x00010053	进卡错误 (53)
0x0001005E	排卡错误 (5E)
0x0001005F	卡片用完 (5F)
0x00010061	认证错误 (61)
0x00010062	认证错误 (62)
0x00010063	认证错误 (63)
0x00010071	转印膜用完 (71)
0x00010072	转印膜错误 (72)
0x00010073	没有放转印膜 (73)
0x00010074	转印膜不支持 (74)
0x00010075	没有放转印膜 (75)
0x00010076	转印膜用完 (76)
0x00010077	转印膜错误 (77)
0x00010078	转印膜错误 (78)
0x00010079	转印膜不支持 (79)
0x0001007A	转印膜安装错误 (7A)
0x000100A1	不是固件支持的命令。
0x000100A2	固件命令的参数错误。
0x000100A3	未创建固件任务。
0x000100A4	固件命令被拒绝。
0x000100A5	打印机内存已满。
0x000100A6	在处理命令之前需要验证。
0x000100A7	固件更新失败。
0x00011001	磁道数据为空。
0x00011002	在磁道 1 中有错误的字符。
0x00011003	在磁道 2 中有错误的字符。
0x00011004	在磁道 3 中有错误的字符。
0x00011005	未接上磁条模块。
0x00011006	写磁条数据失败!
0x00011007	未接上翻转器模块。
0x00011008	600DPI 功能未启用。
0x00011009	第 1 轨是空的。
0x0001100A	第 2 轨是空的。
0x0001100B	第 3 轨是空的。
0x0001100C	第 1,2 轨是空的。
0x0001100D	第 1,3 轨是空的。
0x0001100E	第 2,3 轨是空的。
0x0001100F	第 1,2,3 轨是空的。
0x00011011	色带类型与设置不相符。
0x00011012	未启用打印非标准长度卡的功能。
0x00011013	要打印的文字或图像超过卡的范围。
0x00011014	打印机中没有卡。
0x00011015	请拿出色带。
0x00011020	磁道是空白的。
0x00011021	磁道 1 数据有错误。
0x00011022	磁道 2 数据有错误。
0x00011023	磁道 3 数据有错误。
0x00011024	磁道 1,2 数据有错误。
0x00011025	磁道 1,3 数据有错误。
0x00011026	磁道 2,3 数据有错误。
0x00011027	磁道 1,2,3 数据有错误。

0x00011030	未连接智能卡模块。
0x00011031	校准智能卡位置失败。
0x00011040	固件文件与打印机型号不匹配。
0x00011041	当前固件版本与固件文件相同。
0x00011042	当前固件版本比固件文件新。

6.2 打印机警告代码

打印机警告代码	警告的描述
0x00020001	色带少
0x00020002	少卡 3
0x00020003	等待卡插入
0x00020004	正在等待移除卡
0x00020005	少卡 2
0x00020006	少卡 1
0x00020009	建议清洗程序
0x0002000A	数据不存在
0x00020011	磁条为空
0x00020014	磁条数据错误

6.3 SR30/SR40 智能卡读卡器错误代码

SR30/SR40 错误代码	错误的描述
-1	卡片类型不对
-2	无卡
-3	有卡未上电
-4	卡片无应答
-16	通信错误
-17	超时错误
-32	打开端口错误
-33	获取端口参数错误
-34	设置端口参数错误
-35	关闭端口错误
-36	端口被占用
-48	格式错误
-49	数据格式错误
-50	数据长度错误
-64	读取错误
-65	写入错误
-66	Non_receive 错误
-80	无减少错误
-81	CPU 数据验证错误
-82	RS485 通信时的地址号码错误
-115	获取版本号错误
-194	CPU 卡响应错误
-211	CPU 卡响应超时
-214	CPU 卡验证错误
-215	CPU 卡命令字符错误

1	没有卡或验证错误
2	数据验证错误
3	数值空缺错误
4	身份验证错误
5	校验和验证错误
6	读卡器和卡通通信错误
8	读取卡序列号错误
9	键类型错误
10	卡未验证
11	操作卡位错误
12	操作卡字节错误
15	写卡错误
16	增量操作失败
17	递减操作失败
18	读取卡操作失败
19	传输缓冲区溢出
21	传输帧错误
23	未知传输要求
24	防碰撞错误
25	射频模块复位错误
26	非验证接口
27	射频模块通信超时
60	非正常操作
100	错误的的数据
124	参数值错误

6.4 打印机函数可能返回的 Win32 错误代码

WIN32 错误代码	值	描述
ERROR_INVALID_FUNCTION	1	函数不正确。
ERROR_FILE_NOT_FOUND	2	系统找不到指定的文件。
ERROR_PATH_NOT_FOUND	3	系统找不到指定的路径。
ERROR_TOO_MANY_OPEN_FILES	4	系统无法打开文件。
ERROR_ACCESS_DENIED	5	访问被拒绝。
ERROR_INVALID_HANDLE	6	句柄无效。
ERROR_NOT_ENOUGH_MEMORY	8	没有足够的存储可用于处理此命令。
ERROR_INVALID_ACCESS	12	密码无效。
ERROR_INVALID_DATA	13	数据无效。
ERROR_OUTOFMEMORY	14	没有足够的内存资源来完成此操作。
ERROR_BAD_UNIT	20	系统找不到指定的设备。
ERROR_NOT_READY	21	设备尚未就绪。
ERROR_BAD_COMMAND	22	设备无法识别该命令。
ERROR_BAD_LENGTH	24	该计划发布了一个命令但命令长度不正确。
ERROR_WRITE_FAULT	29	系统无法写入指定的设备。
ERROR_READ_FAULT	30	系统无法从指定的设备读取。
ERROR_GEN_FAILURE	31	连接到系统的设备无法正常工作。
ERROR_NOT_SUPPORTED	50	不支持该请求。
ERROR_PRINTQ_FULL	61	打印机队列已满。

ERROR_PRINT_CANCELLED	63	等待打印的文件已被删除。
ERROR_INVALID_PARAMETER	87	参数不正确。
ERROR_OPEN_FAILED	110	系统无法打开指定的设备或文件。
ERROR_CALL_NOT_IMPLEMENTED	120	此系统不支持此函数。
ERROR_INSUFFICIENT_BUFFER	122	传递给系统调用的数据区域太小。
ERROR_INVALID_NAME	123	文件名、目录名或卷标语法不正确。
ERROR_INVALID_LEVEL	124	系统调用级别不正确。
ERROR_MOD_NOT_FOUND	126	找不到指定的模块。
ERROR_PROC_NOT_FOUND	127	找不到指定的过程。
ERROR_BUSY	170	请求的资源正在使用中。
ERROR_ALREADY_EXISTS	183	该文件已存在时无法创建该文件。
ERROR_NO_DATA	232	管道正在关闭。
ERROR_MORE_DATA	234	有更多的数据可用。
WAIT_TIMEOUT	258	等待操作超时。
ERROR_DEVICE_FEATURE_NOT_SUPPORTED	316	设备不支持命令功能。
ERROR_DEVICE_UNREACHABLE	321	设备无法访问。
ERROR_DEVICE_NO_RESOURCES	322	目标设备没有足够的资源来完成操作。
ERROR_DATA_CHECKSUM_ERROR	323	发生数据完整性校验和错误。文件流中的数据已损坏。
ERROR_OPERATION_IN_PROGRESS	329	设备当前正在进行操作。
ERROR_DEVICE_IN_MAINTENANCE	359	设备处于维护模式。
ERROR_DEVICE_HARDWARE_ERROR	483	由于致命的错误, 请求失败。设备硬件错误。
ERROR_INVALID_ADDRESS	487	尝试访问无效的地址。
ERROR_OPERATION_ABORTED	995	I/O 操作已中止, 因为线程退出或应用程序请求。
ERROR_IO_INCOMPLETE	996	重叠的 I/O 事件未处于终止状态。
ERROR_IO_PENDING	997	正在进行重叠的 I/O 操作。
ERROR_CAN_NOT_COMPLETE	1003	无法完成此功能。
ERROR_BADKEY	1010	配置注册表项无效。
ERROR_BUS_RESET	1111	I/O 总线已重置。
ERROR_IO_DEVICE	1117	由于 I/O 设备错误, 无法执行请求。
ERROR_DLL_NOT_FOUND	1157	找不到运行此应用程序所需的库文件之一。
ERROR_DEVICE_REINITIALIZATION_NEEDED	1164	由于硬件错误, 指定的设备需要重新初始化。
ERROR_DEVICE_REQUIRES_CLEANING	1165	设备已表明在尝试进一步操作之前需要进行清洗。
ERROR_DEVICE_DOOR_OPEN	1166	设备已经表明它的门是打开的。
ERROR_DEVICE_NOT_CONNECTED	1167	设备未连接。
ERROR_BAD_DEVICE	1200	指定的设备名称无效。
ERROR_NO_NETWORK	1222	网络不存在或未启动。
ERROR_CANCELLED	1223	该操作已被用户取消。
ERROR_CONNECTION_REFUSED	1225	远程计算机拒绝了网络连接。
ERROR_NETWORK_UNREACHABLE	1231	无法到达网络位置。
ERROR_HOST_UNREACHABLE	1232	无法到达网络位置。
ERROR_PORT_UNREACHABLE	1234	远程系统上的目标网络终结点没有运行任何服务。
ERROR_REQUEST_ABORTED	1235	请求已中止。
ERROR_CONNECTION_ABORTED	1236	网络连接被本地系统中止。
ERROR_RETRY	1237	无法完成操作。应执行重试。
ERROR_INCORRECT_ADDRESS	1241	网络地址不能用于请求的操作。
ERROR_CONTINUE	1246	继续进行中的工作。
ERROR_NO_MORE_DEVICES	1248	不再有本地设备。

ERROR_MACHINE_LOCKED	1271	计算机已锁定, 且无法关闭, 而没有强制选项。
ERROR_UNIDENTIFIED_ERROR	1287	没有足够的信息来识别失败的原因。
ERROR_DC_NOT_FOUND	1425	无效的设备上下文(DC)句柄。
ERROR_NO_SYSTEM_RESOURCES	1450	没有足够的系统资源来完成请求的服务。
ERROR_TIMEOUT	1460	由于超时期间已过期, 此操作返回。
ERROR_INCORRECT_SIZE	1462	大小参数不正确。
ERROR_FUNCTION_FAILED	1627	函数在执行过程中失败。
ERROR_INVALID_TABLE	1628	指定的表无效或未知。
ERROR_DATATYPE_MISMATCH	1629	提供的数据类型错误。
ERROR_UNSUPPORTED_TYPE	1630	不支持此类型的数据。
RPC_S_NO_ENDPOINT_FOUND	1708	找不到端点。
RPC_S_SERVER_UNAVAILABLE	1722	RPC 服务器不可用。
ERROR_INVALID_USER_BUFFER	1784	所提供的用户缓冲区对所请求的操作无效。
ERROR_UNKNOWN_PORT	1796	指定的端口未知。
ERROR_INVALID_PRINTER_NAME	1801	打印机名称无效。
ERROR_INVALID_PRINTER_COMMAND	1803	打印机命令无效。
ERROR_INVALID_DATATYPE	1804	指定的数据类型无效。
ERROR_INVALID_FORM_SIZE	1903	指定的表单大小无效。
OR_INVALID_OID	1911	找不到指定的对象。
ERROR_NOT_CONNECTED	2250	此网络连接不存在。
ERROR_PRINTER_HAS_JOBS_QUEUED	3009	当有作业排队到打印机时, 不允许执行请求的操作。
ERROR_PRINTER_NOT_FOUND	3012	未找到打印机。
WSAECONNREFUSED	10061	无法进行连接, 因为目标计算机主动拒绝。